

cl-freelock Manual

Andrew D. France

Last edited: 2026-05-28T23:31:51-05:00 UTC

Contents

1	Tutorial	5
1.1	Getting Started: Installation	5
1.2	Basic Usage	5
1.2.1	MPMC Unbounded Queue (Michael-Scott)	6
1.2.2	MPMC Bounded Queue (Vyukov)	7
1.2.3	SPSC Bounded Queue	8
1.2.4	Error Handling	8
2	API Reference	9
2.1	Atomic Primitives	9
2.1.1	ATOMIC-REF	9
2.1.2	MAKE-ATOMIC-REF	9
2.1.3	ATOMIC-REF-VALUE	10
2.1.4	CAS	10
2.1.5	ATOMIC-INCF	10
2.1.6	MEMORY-BARRIER	11
2.2	MPMC Unbounded Queue	11
2.2.1	QUEUE	11
2.2.2	MAKE-QUEUE	11
2.2.3	QUEUE-PUSH	12
2.2.4	QUEUE-POP	12
2.2.5	QUEUE-EMPTY-P	13
2.3	MPMC Bounded Queue	13
2.3.1	BOUNDED-QUEUE	13
2.3.2	MAKE-BOUNDED-QUEUE	14
2.3.3	BOUNDED-QUEUE-CAPACITY	14

2.3.4	BOUNDED-QUEUE-PUSH	14
2.3.5	BOUNDED-QUEUE-POP	14
2.3.6	BOUNDED-QUEUE-EMPTY-P	15
2.3.7	BOUNDED-QUEUE-FULL-P	15
2.3.8	BOUNDED-QUEUE-PUSH-BATCH	15
2.3.9	BOUNDED-QUEUE-POP-BATCH	15
2.4	SPSC Bounded Queue	16
2.4.1	SPSC-QUEUE	16
2.4.2	MAKE-SPSC-QUEUE	16
2.4.3	SPSC-PUSH	16
2.4.4	SPSC-POP	16
3	Performance Guide	17
3.1	General Performance Principles	19
3.1.1	Choose the Right Data Structure	19
3.1.2	Capacity Selection	19
3.1.3	Thread Contention	19
3.2	Optimization Techniques	19
3.2.1	Batch Operations	19
3.2.2	Avoid Busy Waiting	19
3.2.3	Memory Allocation	19
3.3	Implementation-Specific Notes	19
3.3.1	SBCL	19
3.3.2	CLASP	19
3.4	Common Pitfalls	19
3.4.1	Wrong capacity	19
3.4.2	Thread safety	19
3.4.3	Busy waitings	19

3.4.4	Memory leaks	19
3.4.5	False sharing	19
3.5	Recommended Patterns	19
3.5.1	Producer-Consumer Pool	19
3.5.2	Basic Producer-Consumer	19
3.5.3	Bounded Buffer with Batch Processing	19
3.5.4	SPSC Pipeline	19
3.5.5	Load Balancing with Multiple Queues	19
3.5.6	Web Server Request Queue	19
3.5.7	Atomic Counter Service	19
3.5.8	Event Processing Pipeline	19
3.5.9	Memory Pool with Lock-Free Allocation	19

1 Tutorial

1.1 Getting Started: Installation

Once available on quicklisp, you will be able to simply run:

```
(ql:quickload :cl-freelock)
```

For now, clone the repository into your Quicklisp local-projects directory:

```
cd ~/quicklisp/local-projects/  
git clone https://github.com/ItsMeForLua/cl-freelock.git
```

Then, load the system in your REPL:

```
(ql:quickload :cl-freelock)
```

1.2 Basic Usage

In this section, we will go over the basic usage of unbounded Multi-Producer Multi-Consumer (MPMC), bounded MPMC, bounded Single Producer Single Consumer (SPSC), and error handling.

Additional Information

1. `cl-freelock` allows the use of `fl` when calling its functions.

For example: `(fl:queue-push *work-queue* i)`

2. The API reference for `cl-freelock` is mostly written with SBCL in-mind. However, the differences between implementations regarding the code presented in this documentation will be quite minor.

1.2.1 MPMC Unbounded Queue (Michael-Scott)

The algorithm used for unbounded queues is based off of the Michael-Scott algorithm. We specifically use Compare-And-Swap (CAS) operations to link nodes and swing the head and tail pointers.

We identify three main reasons one would use this particular queue type:

1. When the workload is highly variable in predictability, such that you cannot guarantee a maximum capacity for your queue ahead of time.
2. You have a thread pool architecture with multiple producer threads that are throwing jobs onto the queue, and multiple worker threads are simultaneously pulling jobs off. i.e., An N-to-N architecture.
3. If your project requires immunity from deadlocks.

Producers A producer thread works by calling `(queue-push *work-queue* item)`. Because the producer thread is lock-free, the procedure will never block a thread. An example of defining a producer:

```
(defun producer ()  
  (loop for i from 1 to 1000 do  
    (fl:queue-push *work-queue* i)  
    (sleep 0.001)))
```

n.b., It is very important you consider the reasoning for adding a sleep in this example. The sleep is just to simulate the time it may take to fetch the next item. It is not needed in the procedure within normal use-cases; however, if you simply ran the example in a fresh REPL, you would risk locking up your entire environment.^{1, 2}

¹This is called “CPU Starvation”, which is when a procedure consumes 100% of a systems CPU core.

²Concurrent programming is a matter of managing the interleaving of different processes as they interact with a shared state — hence, if we ran a loop from i to 1000 without sleep, most modern CPUs would actually be so fast that the producer may finish pushing all 1000 items before a consumer thread has a chance to wakeup.

Consumers A consumer thread works by calling `(queue-pop *work-queue*)`. This procedure does not return lisp conditions; instead, `queue-pop` returns two specific values:

1. The `item` itself or `NIL` if empty.
2. A `success` boolean, `T` if an item was grabbed, `NIL` if the queue was empty.

n.b., If a queue happens to be empty, then the consumer thread should handle it gracefully by briefly sleeping before polling again. The sleep is a small number, which you set to prevent busy waiting.³

You must wrap the procedure in a `multiple-value-bind` to check if the pop was successful.

```
(defparameter *work-queue* (make-queue))
;; Producer thread
(defun producer ()
  (loop for i from 1 to 1000 do
    (queue-push *work-queue* i)
    (sleep 0.001)))

;; Consumer thread
(defun consumer ()
  (loop
    (multiple-value-bind (item found)
      (queue-pop *work-queue*)
      (if found
        (format t "Processing: ~A~%" item)
        (sleep 0.001)))))
```

1.2.2 MPMC Bounded Queue (Vyukov)

```
(defparameter *buffer* (make-bounded-queue 64)) ; Must be power of 2

;; Non-blocking operations
(when (bounded-queue-push *buffer* "data")
  (format t "Push succeeded~%"))
```

³Busy waiting (or spinning) is when a consumer constantly asks an empty queue if it has any data in queue — consequently, this tanks a systems performance.

```
(multiple-value-bind (value success)
  (bounded-queue-pop *buffer*)
  (when success
    (format t "Got: ~A~%" value))))
```

1.2.3 SPSC Bounded Queue

```
(defparameter *spsc* (make-spsc-queue 256)) ; Must be power of 2
;; Producer thread only
(when (spsc-push *spsc* "fast-data")
  (format t "Pushed successfully~%"))
;; Consumer thread only
(multiple-value-bind (value success)
  (spsc-pop *spsc*)
  (when success
    (format t "Got: ~A~%" value))))
```

1.2.4 Error Handling

Operations return multiple values to indicate success/failure rather than signaling conditions:

```
;; Always check the second return value
(multiple-value-bind (value success)
  (queue-pop *queue*)
  (if success
    (process value)
    (handle-empty-queue)))
```

2 API Reference

The API Reference for `cl-freelock` is a dictionary of all global functions, involving unbounded Multi-Producer Multi-Consumer (MPMC), bounded MPMC, bounded Single Producer Single Consumer (SPSC), error handling, and atomic primitives.⁴

Additional Information

1. `cl-freelock` allows the use of `fl` when calling its functions.

For example: `(fl:queue-push *work-queue* i)`

2. The API reference for `cl-freelock` is mostly written with *SBCL* in-mind. However, the differences between implementations regarding the code presented in this documentation will be quite minor.

2.1 Atomic Primitives

2.1.1 ATOMIC-REF

Signature:

`atomic-ref`

Description:

A structure which holds a value that can be atomically updated—it is the foundational atomic pointer for `cl-freelock`.

2.1.2 MAKE-ATOMIC-REF

Signature:

`(make-atomic-ref value)`

Description:

⁴*n.b.*, The difference between atomic primitives and atomic operations is a matter concerning layers of abstraction—operations are composed of combinations of primitives and non-primitive procedures.

Constructor for the `atomic-ref` structure—creates a new atomic reference that holds the provided `value`.

2.1.3 ATOMIC-REF-VALUE

Signature:

```
(atomic-ref-value atomic-ref)
```

Description:

Accessor for the value held within an `atomic-ref`—used internally by Compare and Swap (CAS) operations.^{5, 6}

2.1.4 CAS

Signature:

```
(cas place old new)
```

Description:

A CAS macro—atomically stored `new` in `place` if the current value of `place` is equal to `old`.

Returns:

The old value of `place`.

2.1.5 ATOMIC-INCF

Signature:

```
(atomic-incf atomic-ref &optional (delta 1))
```

⁵An accessor (*i.e.*, A structure or slot accessor) is a procedure which admits access to an object's internal state.

⁶*n.b.*, `atomic-ref-value` is internally a `defstruct`. In *Common Lisp*, the use of `defstruct` acts as both a getter and setter by the implementation's compiler. A getter is a procedure which specifically returns a read-only value.

Description:

Atomically increments the value of `atomic-ref` by `delta`. It is implemented via a CAS loop to facilitate compatibility across different *Lisp* implementations.

Returns:

The new incremented value.

2.1.6 MEMORY-BARRIER

Signature:

```
(memory-barrier)
```

Description:

Executes a memory barrier—ensures that all memory operations before the barrier are both completed and globally visible before any subsequent operations are allowed to begin.

2.2 MPMC Unbounded Queue

2.2.1 QUEUE

Signature:

```
queue
```

Description:

A lock-free, multi-producer, multi-consumer (MPMC) unbounded queue class based on the MichaelScott algorithm.

2.2.2 MAKE-QUEUE

Signature:

```
(make-queue)
```

Description:

Creates an unbounded queue instance.

Returns:

A fresh `queue` object—initialized with a dummy node.

Example:

```
(defparameter *work-queue* (fl:make-queue))
```

2.2.3 QUEUE-PUSH

Signature:

```
(queue-push queue object)
```

Description:

Pushes an `object` into the unbounded queue.

Returns:

`T` upon a successful push.

2.2.4 QUEUE-POP

Signature:

```
(queue-pop queue)
```

Description:

Pops an object from the unbounded queue.

Returns:

1. The `object` itself, or `NIL` if the queue is empty.
2. A success boolean, `T` if an item was grabbed, `NIL` if the queue was empty.

n.b., These two values must be captured with the `multiple-value-bind`.

2.2.5 QUEUE-EMPTY-P

Signature:

```
(queue-empty-p queue)
```

Description:

Checks if the unbounded queue is empty.

Returns:

`T` if empty, and `NIL` if not.

Warning: In a highly concurrent environment, the state of a queue can change immediately after this call returns, so do not rely on it for flow control. If needed, it is best to rely on the success boolean returned by `queue-pop`.⁷

2.3 MPMC Bounded Queue

2.3.1 BOUNDED-QUEUE

Signature:

Description:

⁷This is called a Time-of-Check to Time-of-Use (TOCTOU) race condition. This is a race condition that specifically occurs when multiple threads or processes access shared data through concurrent means, but without adequate synchronization. *n.b.*, TOCTOU race conditions are exploitable, and thus are security vulnerabilities. MITRE Corporation [2006](#)

2.3.2 MAKE-BOUNDED-QUEUE

Signature:

Description:

2.3.3 BOUNDED-QUEUE-CAPACITY

Signature:

Description:

2.3.4 BOUNDED-QUEUE-PUSH

Signature:

Description:

2.3.5 BOUNDED-QUEUE-POP

Signature:

Description:

2.3.6 BOUNDED-QUEUE-EMPTY-P

Signature:

Description:

2.3.7 BOUNDED-QUEUE-FULL-P

Signature:

Description:

2.3.8 BOUNDED-QUEUE-PUSH-BATCH

Signature:

Description:

2.3.9 BOUNDED-QUEUE-POP-BATCH

Signature:

Description:

2.4 SPSC Bounded Queue

2.4.1 SPSC-QUEUE

Signature:

Description:

2.4.2 MAKE-SPSC-QUEUE

Signature:

Description:

2.4.3 SPSC-PUSH

Signature:

Description:

2.4.4 SPSC-POP

Signature:

Description:

3 Performance Guide

Tips and guidelines for optimal performance with `cl-freelock` — considerations for the performance of unbounded Multi-Producer Multi-Consumer (MPMC), bounded MPMC, bounded Single Producer Single Consumer (SPSC), error handling, and atomic operations.

Additional Information

1. `cl-freelock` allows the use of `fl` when calling its functions.

For example: `(fl:queue-push *work-queue* i)`

2. The API reference for `cl-freelock` is mostly written with SBCL in-mind. However, the differences between implementations regarding the code presented in this documentation will be quite minor.

3.1 General Performance Principles

3.1.1 Choose the Right Data Structure

3.1.2 Capacity Selection

3.1.3 Thread Contention

3.2 Optimization Techniques

3.2.1 Batch Operations

3.2.2 Avoid Busy Waiting

3.2.3 Memory Allocation

3.3 Implementation-Specific Notes

3.3.1 SBCL

3.3.2 CLASP

3.4 Common Pitfalls

3.4.1 Wrong capacity

3.4.2 Thread safety

3.4.3 Busy waitings

3.4.4 Memory leaks

3.4.5 False sharing

3.5 Recommended Patterns

3.5.1 Producer-Consumer Pool

3.5.2 Basic Producer-Consumer

3.5.3 Bounded Buffer with Batch Processing

3.5.4 SPSC Pipeline